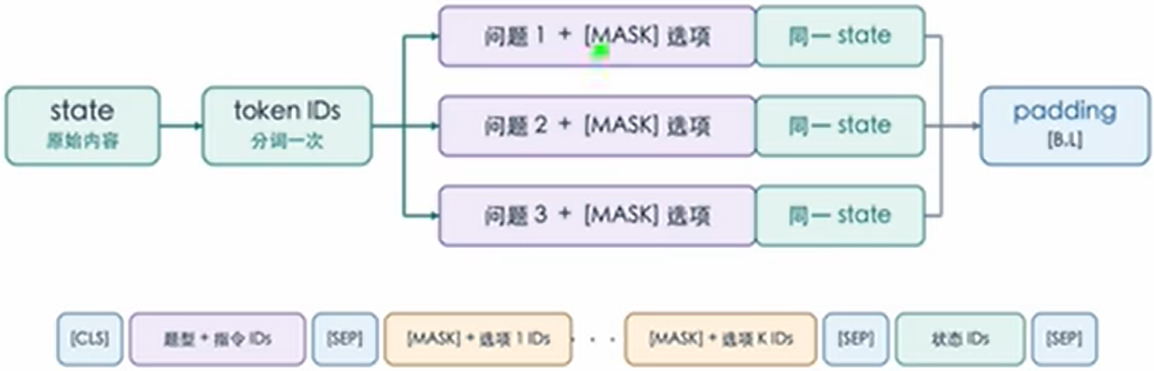
Laya（Jev开源版）

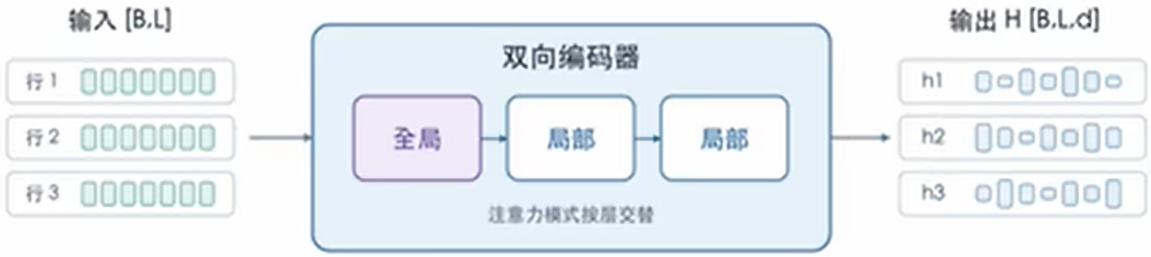
> 🥖 **Laya**

- Laya：
  - Bert 类编码器
    - 非自回归、双向编码器
  - 面向选项的决策头
    - 两层 Transformer
  - 概率训练
    - RLCD（Reinforcement Learning for Calibrated Decisions）
    - 让概率有实际意义
      - 对大量给出约 80% 概率的判断，期望其中约 80% 成立
- 输入输出
  - Choice（选哪一个）
    - 输入：问题和一组无先后顺序的候选项
    - 输出：选中的项，以及每项的概率
  - Score（程度有多高）
    - 输入：问题和从低到高的等级说明
    - 输出：各等级的概率，以及按这些概率算出的综合分数
  - Noul（是不是）
    - 输入：一个是非问题，是和否的判定标准
    - 输出：是的概率

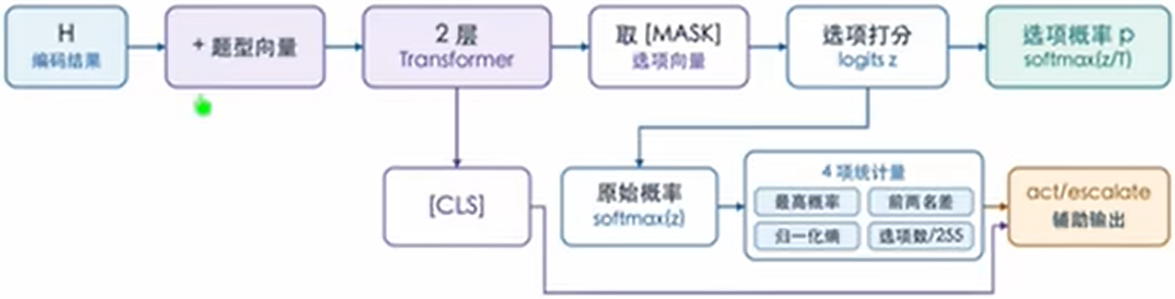
数据处理与输入



编码器



决策头与输出



- 题型向量
  - 给本题每个 token 的编码向量加上 choice、score 或 noul 的类型向量
  - 让共用的决策头区分任务类型
- 两层 Transformer
  - 对编码结果再做两层上下文交互
  - 使问题、选项和状态的信息进一步结合
- 取 [MASK] 向量
  - 提取每个选项前的 [MASK] 表示
  - 得到一题中各候选项的向量
- 选项打分
  - 每个选项向量经过 `LayerNorm → Linear(d,d) → GELU → Linear(d,1)`
  - 产生每个选项的 logits
- 选项打分
  - 每个选项向量经过 `LayerNorm → Linear(d,d) → GELU → Linear(d,1)`
  - 产生每个选项的 logits
- 选项概率
  - 计算 `softmax(z/T)`
  - 得到每个选项的概率
- 辅助头
  - 将决策头输出的 [CLS] 向量与原始概率的四项统计量拼接
  - 产生 act/escalate 的两个 logits

代码块 Python 复制

```python
def forward(self, input_ids, attention_mask, marker_pos, marker_mask, c
    h = self.encoder(
        input_ids=input_ids,
        attention_mask=attention_mask,
    ).last_hidden_state

    h = h + self.type_emb(qtype)[:, None, :]
    h = self.head(h, src_key_padding_mask=~attention_mask.bool())

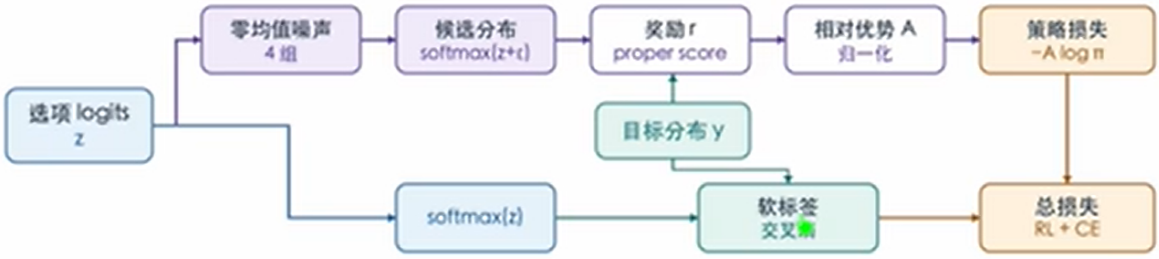
    index = marker_pos.unsqueeze(-1).expand(-1, -1, h.size(-1))
    option_vectors = torch.gather(h, dim=1, index=index)

    logits = self.scorer(option_vectors).squeeze(-1).float()
    logits = logits.masked_fill(~marker_mask, -1e4)

    raw_probs = torch.softmax(logits.detach(), dim=-1)
    statistics = option_statistics(raw_probs, marker_mask)
    act_input = torch.cat([h[:, 0].float(), statistics], dim=-1)
    act_logits = self.act_head(act_input)

    return logits, act_logits
```

训练算法（RLCD）



设一道二选一的 choice 题：

- 模型输出 $z = (1.2, 0.4)$
- 原始概率为 $(0.690, 0.310)$
- 训练目标是 $y = (0.85, 0.15)$

- 加噪
  - 作用：在当前预测附近试探几个方向，让策略项知道往哪个方向调整更靠近目标分布
  - 采样 4 组随机噪声，每组噪声减去自身均值
    - 假设四个噪声向量为：(0.55,-0.15), (0.02,-0.22), (-0.21,-0.05), (-0.42,0.08)
    - 分别减去自身均值得到：(0.35,-0.35), (0.12,-0.12), (-0.08,0.08), (-0.25,0.25)
    - 对同一条样本的 z 加四组不同噪声，分别计算 $softmax(z + \epsilon_g)$，得到四个候选分布：(0.818,0.182), (0.739,0.261), (0.655,0.345), (0.574,0.426)

- 奖励和优势

$$r_g = \underbrace{\sum_i y_i \log q_{g,i}}_{\text{对数得分}} + 0.75 \underbrace{\frac{y \cdot q_g}{\|q_g\|_2}}_{\text{球面得分}} - \mathbf{1}_{\text{score}} \underbrace{\text{RPS}(q_g, y)}_{\text{有序等级误差}}$$

  - 奖励组成
    - 对数得分：鼓励给目标分布中占比较大的选项足够的概率，也会惩罚过度自信
    - 球面得分：比较预测向量与目标向量的方向是否接近，进一步奖励匹配程度更高的分布
    - RPS：只用于有顺序的 score 等级，比如真实等级为 3 时，把概率放在等级 2 比放在等级 0 错得轻
  - 与 y 比较，4 个候选分布得到 4 个奖励 (0.220, 0.180, 0.097, -0.020)
  - 四个奖励的均值约为 0.119，减去均值后得到 (0.101, 0.061, -0.022, -0.139)
  - 减去均值除以标准差得到优势
  - 标准差是当前批次所有中心化奖励的标准差，不是每道题计算一个标准差
- 交叉熵
  - 对未加噪的 `softmax(z)` 与目标 y 计算软标签交叉熵，让原始预测直接靠近目标
  - 总损失为策略损失加交叉熵损失

代码块 Python 复制

```python
def rlcd_loss(logits, target, marker_mask, qtype, sigma):
    mask = marker_mask.bool()
    k = mask.sum(dim=-1, keepdim=True).float()

    noise = torch.randn((4,) + logits.shape, device=logits.device) * si
    noise = noise * mask
    noise = (noise - noise.sum(dim=-1, keepdim=True) / k) * mask

    samples = logits.detach().unsqueeze(0) + noise
    probs = torch.softmax(samples.masked_fill(~mask, -1e4), dim=-1) * m

    with torch.no_grad():
        log_score = (
            target * probs.clamp_min(1e-12).log().clamp_min(-9.21)
        ).sum(dim=-1)
        spherical = (target * probs).sum(dim=-1) / probs.norm(
            dim=-1
        ).clamp_min(1e-9)

        rps = (
```

```
    with torch.no_grad():
        log_score = (
            target * probs.clamp_min(1e-12).log().clamp_min(-9.21)
        ).sum(dim=-1)
        spherical = (target * probs).sum(dim=-1) / probs.norm(
            dim=-1
        ).clamp_min(1e-9)

        rps = (
            ((probs.cumsum(-1) - target.cumsum(-1)) ** 2 * mask)
            .sum(dim=-1)
            / (k.squeeze(-1).clamp_min(2) - 1)
        )
        # 加上题型为 score 时, qtype = 1
        reward = log_score + 0.75 * spherical - (qtype == 1) * rps

        advantage = reward - reward.mean(dim=0, keepdim=True)
        advantage = advantage / (advantage.std() + 1e-6)
```

```
        advantage = reward - reward.mean(dim=0, keepdim=True)
        advantage = advantage / (advantage.std() + 1e-6)

    log_policy = -(
        ((samples - logits.unsqueeze(0)) ** 2 * mask).sum(dim=-1)
        / (2 * sigma**2)
    )
    loss_policy = -(advantage * log_policy).mean()

    log_probs = torch.log_softmax(logits.masked_fill(~mask, -1e4), di
    loss_ce = -(target * log_probs).sum(dim=-1).mean()
    return loss_policy + loss_ce
```